Technical Documentation

1 Database Requirement

1.1 User and System Requirement

1. User Requirements:

- Employees:
  - View, update, insert, and delete student details
  - Add, view, update, and delete events.
  - Manage and view donor details.
  - View donation details.
- Donors:
  - Make donations and view their donation history.
  - View upcoming events.
  - View which donors attending specific events.

2. System Requirements:

- Sensitive data are secured not everyone can see these data.
- The system is user-friendly allowing users to access the system and interact with it effectively.
- The system is scalable, it can support additional donors, events, employees, students, and donations.

1.2 Data Requirements

- An employee manages multiple students and donors, each donor has a name, contact information such as email, and career details such as the organization they work for.
- Each donor can attend multiple events and donate a multiple donation while the events can be attended by one or more donor.
- Each donation made has a cost and currency.
- It is required to keep track of the events which the employees organize.
- Each employee has a name and may have a multiple email and phone.

2 Database Design

2.1 Conceptual Design

In the conceptual design I was successfully able to collect requirement and analyse the problem space, identify entities and relationships, draw the model.

The conceptual design I made included entities like donor, employee, event, student, and donation. I choose donor because the whole database is built on the donors, employee because the employee will manage the students, the donors, and organize the events, event because there where will all funds and the donors will attend these events, donation because the fund

value and currency are important to know in these foundations, and students because they are the main beneficiaries from the funds.

Relations:

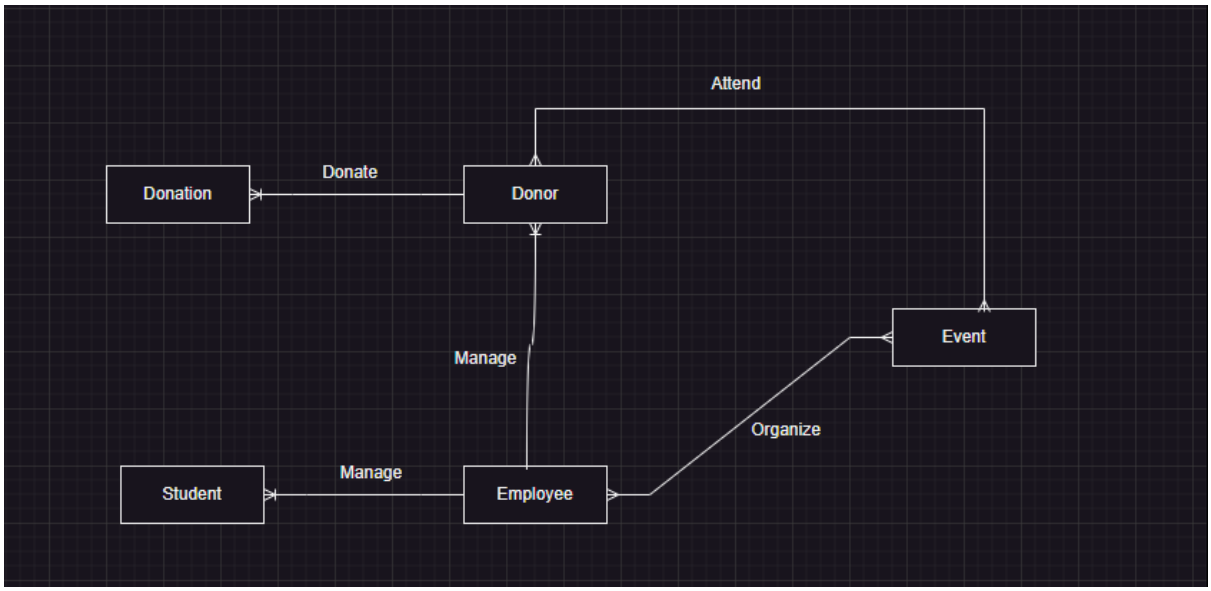
Employee with donor relation is one to many because one employee manages multiple donors, and one donor is managed by one employee.

Employee with student relation is one to many because one employee manages multiple students, and one student is managed by one employee.

Employee with event relation is many to many because an employee organizes multiple events, and an event is organized by one or more employee.

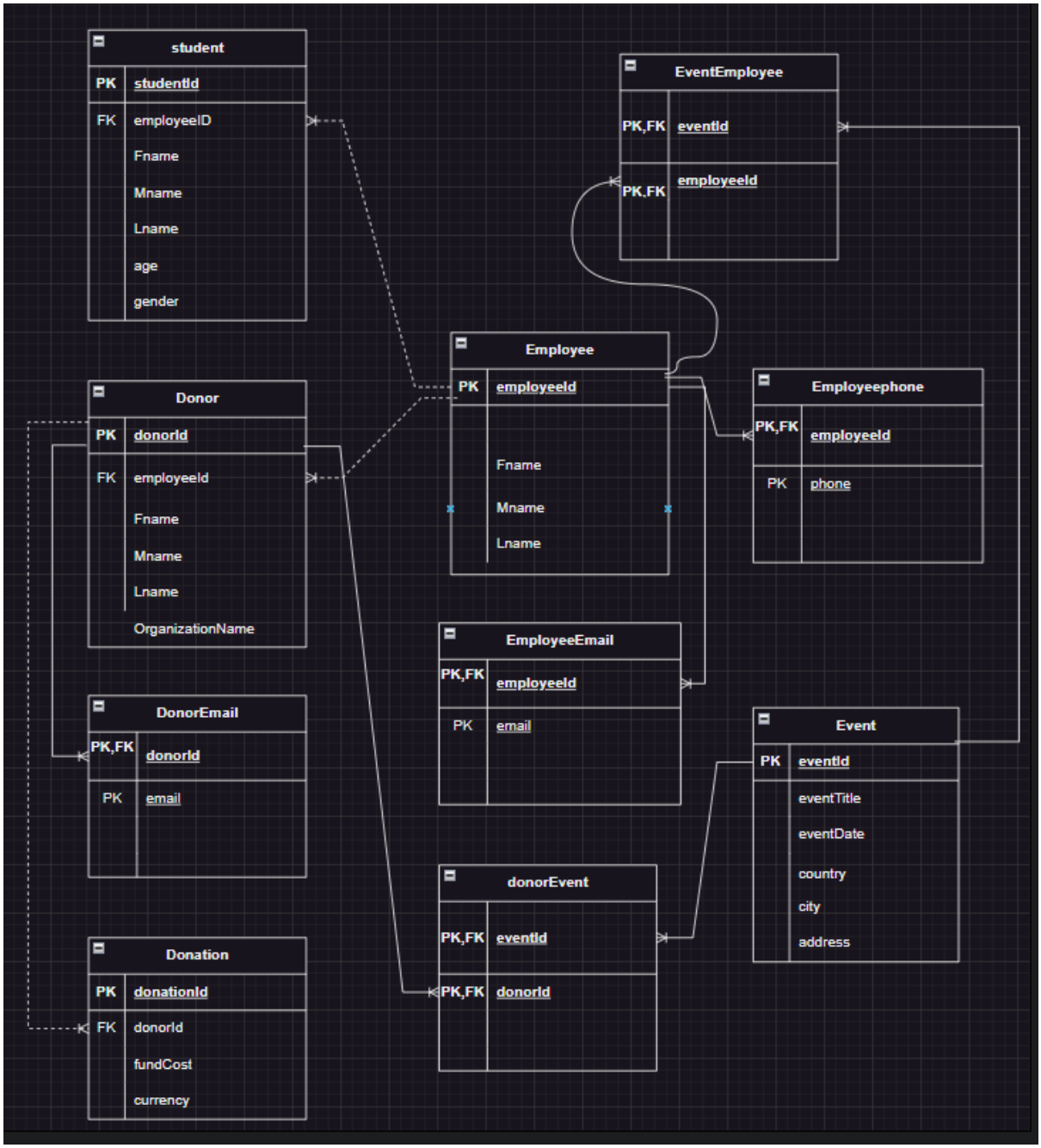
Donor with event relation is many to many because a donor can attend multiple events, and an event can be attended by one or more donor.

Donor with donation relation is one to many because a donor can make multiple donations, and one donation is donated by one donor.



2.2 Logical Design

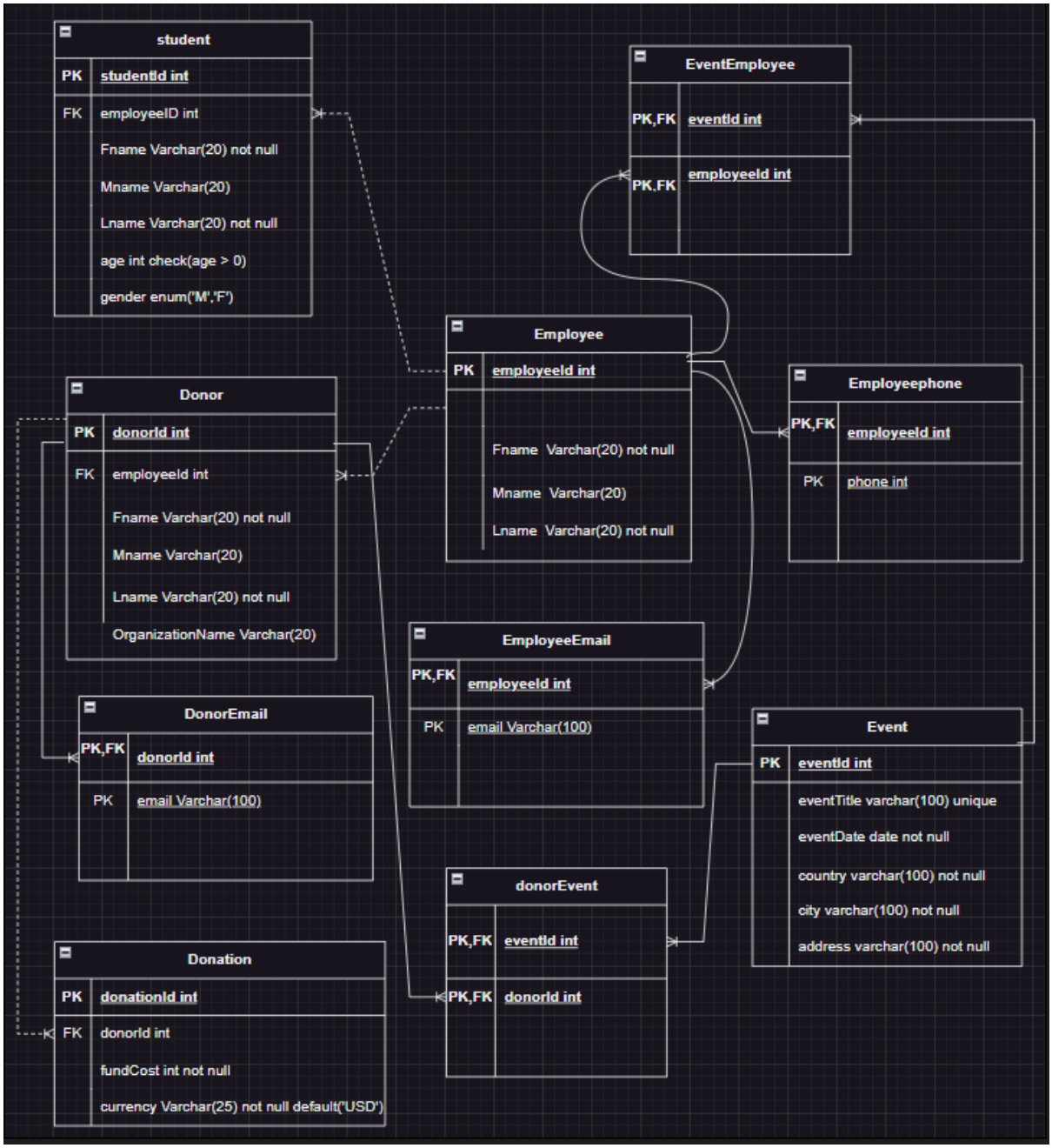
At first in the conceptual design, I only had 5 entities which they are employee, donor, donation, event, and student. But I had multivalued attributes at the employee and donor entities which they are email, and phone, so I made a new three tables EmployeeEmail, EmployeePhone, and DonorEmail. Also, I had two many to many relationships between donor with event and event with employee, So I made new two tables donorEvent and EventEmploye. Also I had composite attributes which made me fix these attributes from one attribute ‘name’ to three attributes ‘Fname, Sname, Lname’ in three entities donor, employee, and student. Also Event entity had a composite attribute which is ‘eventLoc’ and I divided it into ‘country, city, address so from 5 tables in the conceptual design the model transformed to 10 tables in the logical design.



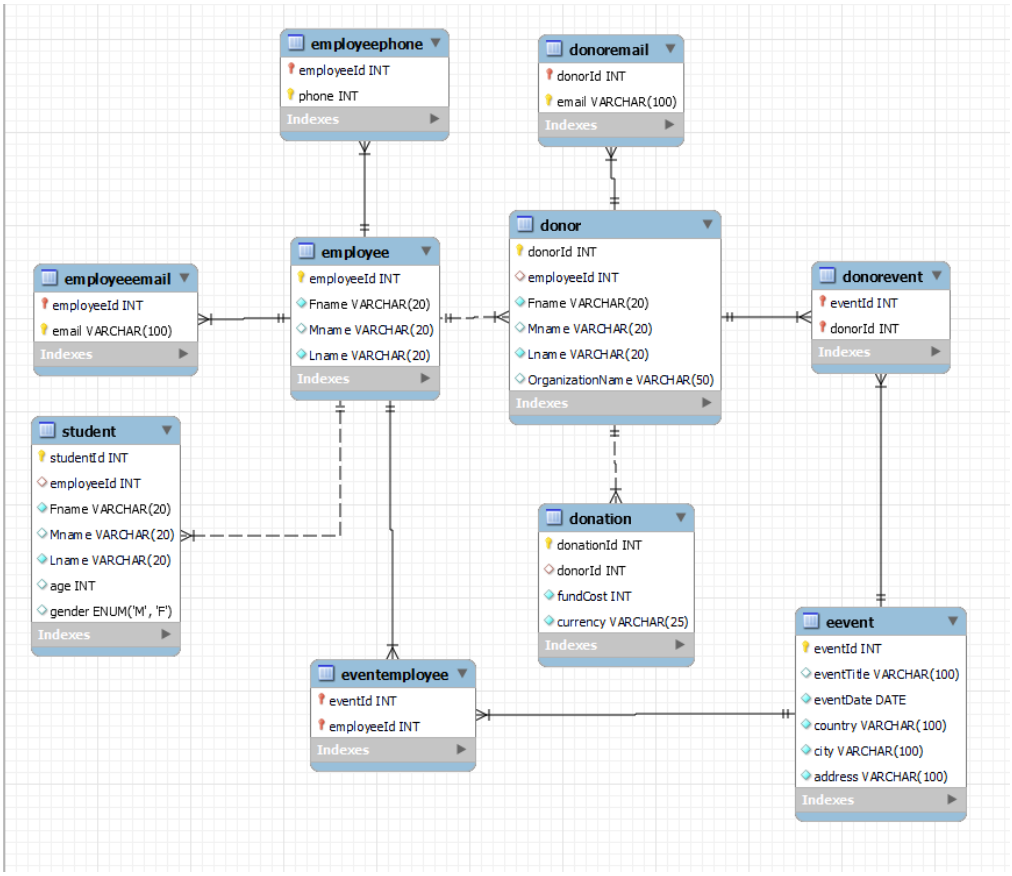
2.3 Physical Design

In the physical design I added the data type of each attribute and the constraints of the attributes.

The rest of the attributes datatypes and constraints are shown in the figure below

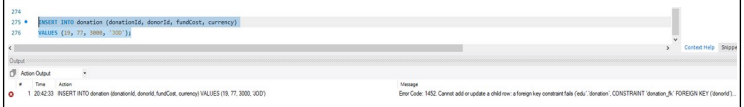
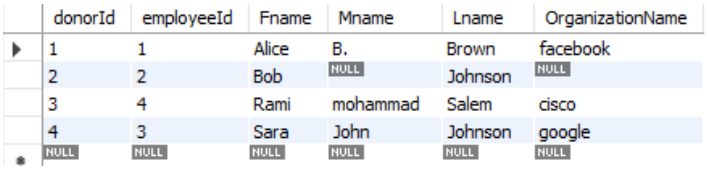


3 Physical Schema

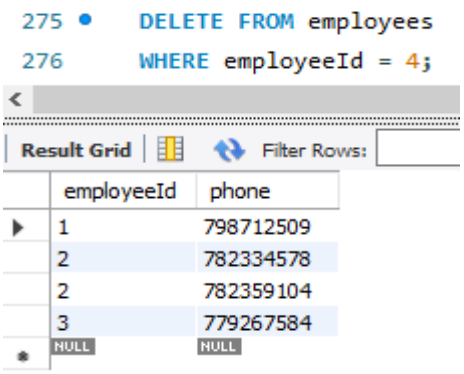
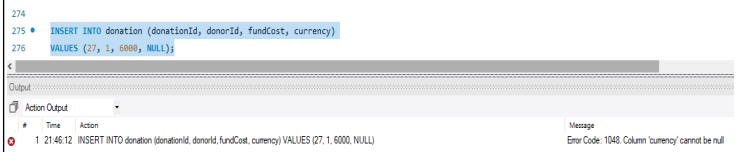


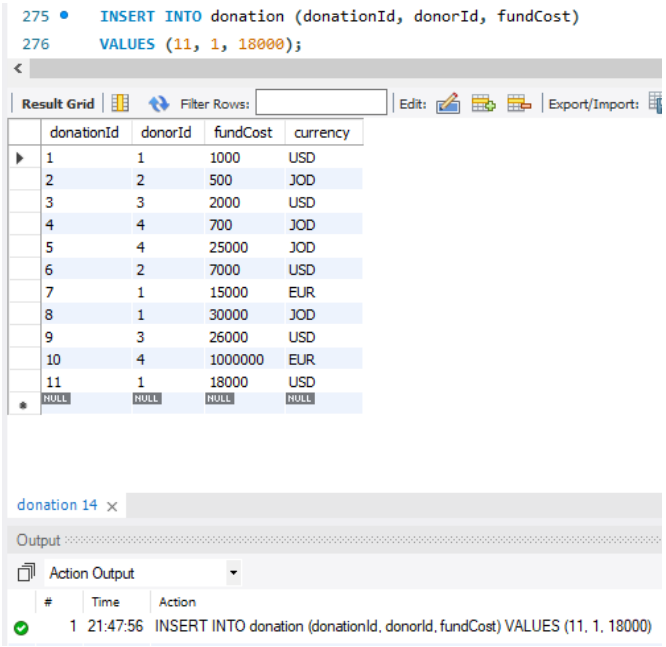
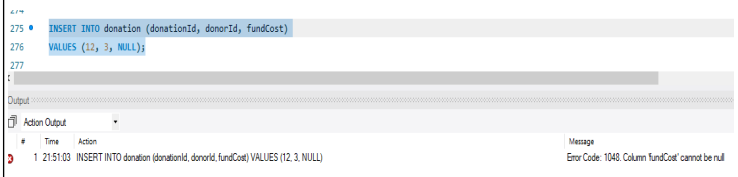
4 Testing

4.1 Data Validation

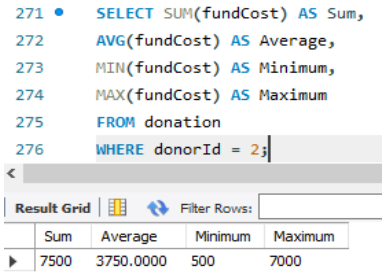
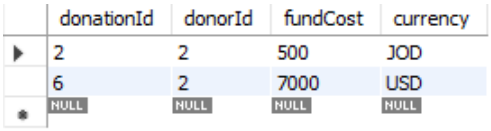
Number	Type	Description	screenshot
1.	All cases of PK	The primary key should not accept null values, in this test case I will try to insert a null value into the primary key.	
		The primary key should be unique, so in this case I will try to insert a duplicate value in the primary key field.	
2.	All cases of FK	The foreign key should be exists as a primary key In another table, so in this test	
		If the primary key is updated the foreign key should update the value.	Before: 

		The primary key is updated from 4 to 10 so the foreign key is also updated.	After: 282 • UPDATE employee 283 SET employeeId = 10 284 WHERE employeeId = 4; 285 286 • select * from donor; Result Grid | Filter Rows: | Edit: | Export donorId, employeeId, Fname, Mname, Lname, OrganizationName 1, 1, Alice, B., Brown, facebook 2, 2, Bob, NULL, Johnson, NULL 3, 10, Rami, mohammad, Salem, cisco 4, 3, Sara, John, Johnson, google
		If the primary key is deleted the foreign key should set the value to NULL. The primary key is deleted so the foreign key is set to NULL.	Before: donorId, employeeId, Fname, Mname, Lname, OrganizationName 1, 1, Alice, B., Brown, facebook 2, 2, Bob, NULL, Johnson, NULL 3, 4, Rami, mohammad, Salem, cisco 4, 3, Sara, John, Johnson, google After: 275 • DELETE FROM employee 276 WHERE employeeId = 4; Result Grid | Filter Rows: | Edit: | Export donorId, employeeId, Fname, Mname, Lname, OrganizationName 1, 1, Alice, B., Brown, facebook 2, 2, Bob, NULL, Johnson, NULL 3, NULL, Rami, mohammad, Salem, cisco 4, 3, Sara, John, Johnson, google
		In another tables, if the primary key is deleted the foreign key row should be deleted. The primary key is deleted so the foreign key row is deleted	Before: employeeId, phone 3, 779267584 2, 782334578 2, 782359104 4, 798355281 1, 798712509 NULL, NULL After:

			
3.	Unique	The eventTitle is UNIQUE so I will test on this attribute. It should not be able to record duplicate names.	
4.	Default	The currency is default with USD. If I set it to NULL it cant be NULL.	

Default		The currency is default with USD, If I didn't put a value for it, it will be by default USD.	
5.	Not null	I will try to insert a NULL value into a NOT NULL attribute "fundCost".	
6.	Check	Age attribute has a check if age greater than 0, I will try to insert a value smaller than 0.	

4.2 Output Validation

Number	Query Description	Screenshot (query + result)	Result validation
1.	A query that retrieves the sum, average, minimum, maximum of donations made by the donor that his id is 2.		This is the donations made by donorId = 2, if we do the summation, average, minimum, maximum calculations, the result will be as the result I provided. 

<table>
<tr>
<td>2.</td>
<td>A query that retrieves the data from two tables, from the employee table it will retrieves employeeId and name who’s managing students, from the student table it will retrieve the studentId and Name.</td>
<td>

```
SELECT
employee.employeeId,
concat(employee.Fname, ' ', employee.Lname) AS employeeName,
student.studentId,
concat(student.Fname, ' ', student.Lname) AS studentName
FROM employee
INNER JOIN student ON
employee.employeeId = student.employeeId;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employeeId	employeeName	studentId	studentName
1	John Doe	1	Chris Evans
2	Jane Smith	2	Emma Watson
3	Mark Ahmad	3	Mohammad mishington
3	Mark Ahmad	4	vito corllion
4	Mohammad Dghimat	5	Sara White

</td>
<td>If we match values from the students and the employee tables it will be as the query result.

Student:

studentId	employeeId	Fname	Mname	Lname	age	gender
1	1	Chris	C.	Evans	21	M
2	2	Emma	NULL	Watson	18	F
3	3	Mohammad	mark	mishington	17	M
4	3	vito	moe	corllion	23	M
5	4	Sara	Walter	White	19	F
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Employee:

employeeId	Fname	Mname	Lname
1	John	Moe	Doe
2	Jane	NULL	Smith
3	Mark	rashed	Ahmad
4	Mohammad	Fathi	Dghimat
NULL	NULL	NULL	NULL

</td>
</tr>
<tr>
<td>3.</td>
<td>Multiple queries that insert, delete, update data from a table</td>
<td>The table after I apply the queries:


```
INSERT INTO donation(donationId, donorId, fundCost, currency)
VALUES (12, 2, 5000, 'JOD');
DELETE FROM donation
WHERE donationId = 22;
UPDATE donation
SET fundCost = 1000
WHERE donationId = 4;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

donationId	donorId	fundCost	currency
1	1	1000	USD
2	2	500	JOD
3	3	2000	USD
4	4	1000	JOD
5	4	25000	JOD
6	2	7000	USD
7	1	15000	EUR
8	1	30000	JOD
9	3	26000	USD
10	4	1000000	EUR
12	2	5000	JOD
NULL	NULL	NULL	NULL

</td>
<td>The table before I apply the queries:

donationId	donorId	fundCost	currency
1	1	1000	USD
2	2	500	JOD
3	3	2000	USD
4	4	700	JOD
5	4	25000	JOD
6	2	7000	USD
7	1	15000	EUR
8	1	30000	JOD
9	3	26000	USD
10	4	1000000	EUR
22	3	4000	JOD
NULL	NULL	NULL	NULL

</td>
</tr>
<tr>
<td>4.</td>
<td>A query that retrieves the sum, average, minimum, maximum, total num of donations made by each donor.</td>
<td>

```
SUM(fundCost) AS SumOfDonations,
AVG(fundCost) As AvgOfDonations,
MIN(fundCost) AS MinimumDonation,
MAX(fundCost) AS MaximumDonation,
COUNT(fundCost) AS CountDonations
FROM donation
GROUP BY donorId
ORDER BY donorId ASC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

donorId	SumOfDonations	AvgOfDonations	MinimumDonation	MaximumDonation	CountDonations
1	46000	15333.3333	1000	30000	3
2	12500	4166.6667	500	7000	3
3	28000	14000.0000	2000	26000	2
4	1026000	342000.0000	1000	1000000	3

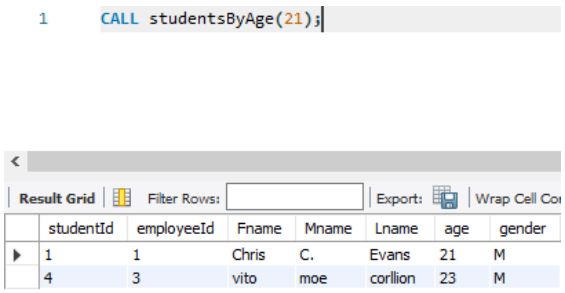
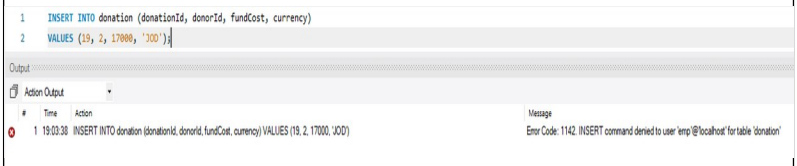
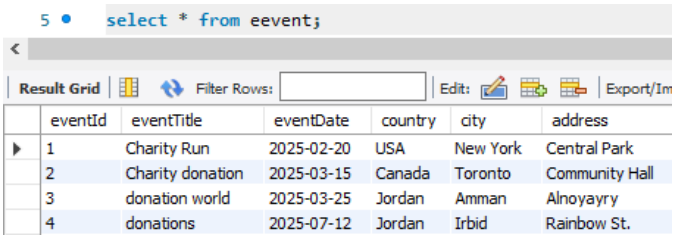
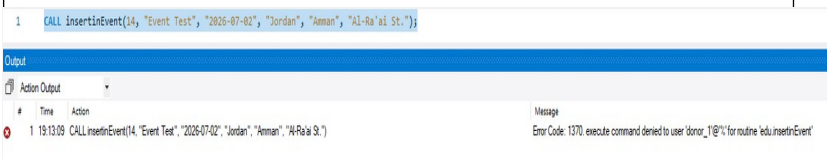
</td>
<td>if we match the data and make calculations, the result will be as the query result.

donationId	donorId	fundCost	currency
1	1	1000	USD
2	2	500	JOD
3	3	2000	USD
4	4	1000	JOD
5	4	25000	JOD
6	2	7000	USD
7	1	15000	EUR
8	1	30000	JOD
9	3	26000	USD
10	4	1000000	EUR
12	2	5000	JOD
NULL	NULL	NULL	NULL

</td>
</tr>
</table>

4.3 Security Validation

Number	Username	Description of privilege/no	Screenshot (query + result)

		privilege	
1.	emp	I gave this user a privilege on the procedure studentsByAge, so we will test if the user can execute the procedure.	
2.	emp	The user dosen't have privilege to insert data in the donation table, so we will test the privilege.	
3.	Donor_1	I gave this user a select privilege on the table eevent, so we will test if the user can view the eevent data.	
4.	Donor_1	The user should not be able to execute the procedeure insertinEvent, so if we tried to execute, an error should happens.	

5 Evaluation of database solution

5.1 Suggested improvements

- Security: As we gave the users a direct permission on the tables by select, insert, update, delete commands, but it is better if we user grant execute permissions on specific stored procedures. This will provide better control over data access and improve the security, it will make users use the functionalities not directly but through these stored procedures.